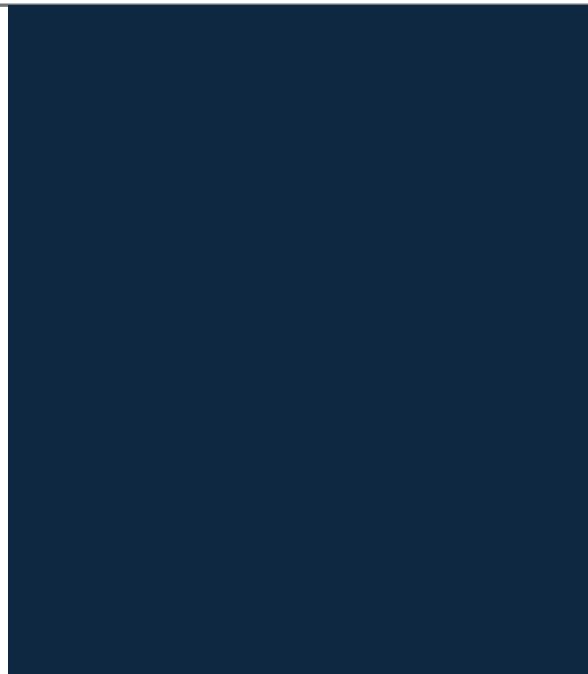




PED V2.0

30-09-2026

Liam Dickson, Steff Zondagh, Ruan
Wolmaranse



Contents

Problem & business need	3
Stakeholder Analysis	3
Scope Baseline	4
In scope	4
Out of Scope	4
Deferred	4
Functional Requirements (FR)	5
1. Requester (FR-REQ)	5
2. Staff (FR-STF)	6
3. Management and Oversight Capabilities (FR-MNG)	8
Non-Functional Requirements (NFR)	10
1. Security Engineering (NFR-SEC)	10
2. Performance, Reliability, and Scalability (NFR-PRS)	11
3. Usability, Accessibility, and Portability (NFR-UAP)	12
4. Environment, Deployment, and Operations (NFR-EDO)	12
Constraints	13
Initial RTM	14
Risk Register	17
Risk Assessment Matrix	17
Architecturally Significant Requirements and Quality Drivers	21
Architecture decision and diagrams	23
Technology-stack decision	24
Data and persistence baseline	24
Mongo DB Schemas	24
Users:	25
Requests:	25
Request_Groups:	27
LOGS	27
Justification and Persistence model:	29
Scalability Strategy (Sharding)	30
Concurrency, Validation, and Transaction Integrity	30
Multi-Document ACID Transactions:	30
Security and Clearance Enforcement:	32
Authorisation Checks	32
User Roles:	32

Field-Level Encryption.....	32
Password Hashing:	32
Traceability artifacts.	32
Initial Design Decisions — Informed by A2 Research	33
Design Decision 1:.....	33
Design Decision 2:.....	35
Deployment Compatibility.....	39
Plausibility of Decisions	39
Configuration, State & Networking.....	40
Deployment Risks	Error! Bookmark not defined.
Deferred Deployment Decisions	41
Realized Issues Log	41
Engineering consideration documentation.....	41
Future engineering considerations	43
Github & Team Governance.....	47
Accountability Architecture	47
Github security policy	49
AI usage and accountability.....	49
AI usage document.....	49
Usage methodology.....	50

Problem & business need

The organisation's biggest problem is there is no controlled system in place for sending, reviewing and completing requests. This results in staff not taking accountability for requests or are unaware of requests that are being sent.

Stakeholder Analysis

<i>Stakeholder Group</i>	<i>Value to project</i>	<i>Stakeholder Priorities</i>	<i>Stakeholder Impact</i>	<i>Plan for Engagement</i>	<i>User Type</i>
Community Members	Low	Stakeholder prioritises request safety and security. They value the privacy of their requests	Failure to ensure buy-in from Community members stands to harm confidence in the organisation as a whole	Engage with communities during the planning phase of the security protocols phase	None
Maintenance crews	High	Easy to grouping of maintenance requests and conjoined notes on requests	Failure to ensure buy-in from maintenance crews risks the failure of the project as a whole, as they are a key stakeholder group	Engage with members of the maintenance crews when developing request control protocols and front end systems for this user group	Maintenance staff
Management	Moderate	Easy and effective traceability of request from creation to resolution including productivity of maintenance crews and backlog tracking	Failure to ensure buy-in from management promotes teams to pursue redevelopment of program instead of evolution.	Engage with members before and after main request and resolution system has been completed to ensure monitoring systems are functional and	Management

Community Engagement Teams				adequate for their needs	
	High	Easy request creation and effective update processes. They also value traceability and accountability for requests	Failure to ensure buy in of Community engagement teams risks issues when migrating from pre-existing systems.	Engage with members of this group when request control protocols are being planned as well as ensuring they are contacted during the implementation phase for front end systems for their user group	Requester

Scope Baseline

In scope

- Create a Request Tracker System that clients can send their requests to and also where staff can log and track those requests
- Analytic dashboard to see statistics of the status of requests and staff's performance.

Out of Scope

- Storing the staff's private information, because it is not needed for the platform to work.

Deferred

- WhatsApp message integration system, where clients can send their requests through WhatsApp and it will be record to the platform. This deferred

Functional Requirements (FR)

1. Requester (FR-REQ)

The following requirements dictate the capabilities of users or administrative requesters submitting and reviewing issues on the platform.

FR-REQ-001: Service Request Submission

<i>Stakeholder Source</i>	Requester
<i>Priority</i>	High
<i>Description</i>	The system must allow a logged-in requester to submit a new service request containing a title, detailed description, location, and contact information.
<i>Acceptance Criteria</i>	➔ Verify that a requester can input text into Title, Description, Location, and Contact Info fields. ➔ Verify that the system validates that all necessary fields are populated before allowing submission. ➔ Verify that upon submitting, the system generates a unique tracking ID and persists the record.

FR-REQ-002: Controlled Category Classification

<i>Stakeholder Source</i>	Requester
<i>Priority</i>	High
<i>Description</i>	The system must enable a requester to categorize their service request using a list of predefined categories.
<i>Acceptance Criteria</i>	➔ Verify that the request form enables a required dropdown selection populated exclusively from the system's available categories. ➔ Verify that a requester cannot submit a service request without selecting exactly one valid category.

FR-REQ-003: Real-Time Status Visibility

<i>Stakeholder Source</i>	Requester
<i>Priority</i>	High
<i>Description</i>	The system must allow a requester to view the current live status of any service request that they have submitted.
<i>Acceptance Criteria</i>	➔ Verify that the requester UI displays the real-time lifecycle status for the selected request, whether it's submitted, assigned, in progress, resolved, or closed. ➔ Verify that the status displayed matches the current record status in the database immediately upon database updates.

FR-REQ-004: Request History

<i>Stakeholder Source</i>	Requester
<i>Priority</i>	Medium
<i>Description</i>	The system must provide a view displaying a historical list of all service requests previously submitted by the logged-in requester.
<i>Acceptance Criteria</i>	➔ Verify that the requester can access a page listing all their requests and full submission history. ➔ Verify that the list is sorted by submission date, showing newest first. ➔ Verify that a requester cannot view service requests submitted by other users, enforcing access control.

FR-REQ-005: Feedback Notifications

<i>Stakeholder Source</i>	Requester
<i>Priority</i>	Medium
<i>Description</i>	The system must generate and deliver feedback to the requester when their request is accepted, rejected, updated, or completed.
<i>Acceptance Criteria</i>	➔ Verify that changing a request's status triggers a notification, through either email or SMS based on their choice, that gets sent to the requester containing the request ID, the new status, and any staff-provided notes within 10 seconds of the status change.

2. Staff (FR-STF)

The following requirements govern the capabilities of authorized staff members responsible for resolving service requests.

FR-STF-001: Role-Based Service Request Filtering

<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must restrict the visibility of service requests so that only staff members authorized, dependent on their role and department, can view requests relevant to them.
<i>Acceptance Criteria</i>	➔ Verify that when a staff member logs in, they can only view service requests matching their assigned role or department. ➔ Verify that attempts to retrieve restricted request logs present an "Access Denied" error message.

FR-STF-002: Advanced Search, Filter, and Sort

<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must provide staff with search, filtering, and sorting capabilities using a selection of options. Options include but are not limited to – Request ID, Category, Status, Priority, Submission Date, and Keyword Search.
<i>Acceptance Criteria</i>	➔ Verify that searching for a valid keyword or unique request ID displays matching results. ➔ Verify that filtering by a combination of options restricts the visible list to only those records. ➔ Verify that sorting by submission date reorders the displayed list correctly.

FR-STF-003: View Request Information

<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must display a comprehensive view of the details of a selected service request to authorized staff, including attachments, current owner of request, and all comments from requester and other maintenance staff.
<i>Acceptance Criteria</i>	➔ Verify that selecting a request loads a detailed page containing all required information, descriptions, historical log entries, and media attachments.

FR-STF-004: Responsibility Assignment and Ownership

<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must allow an authorized staff member to accept, assign or reassign responsibility for an unassigned request.
<i>Acceptance Criteria</i>	➔ Verify that unassigned request displays a “Claim Request” option for staff. ➔ Verify that staff can select from a dropdown list of active, authorized team members to assign the request. ➔ Verify that the system logs the identity of the assigner, assignee, and timestamp in the request’s details.

FR-STF-005: Controlled Status Transitions

<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High

<i>Description</i>	The system must restrict status changes to follow a standardized transition sequence: Submitted – Assigned – In Progress – Resolved – Closed.
<i>Acceptance Criteria</i>	➔ Verify that staff can only transition a request to a valid downstream state. It cannot go from submitted to closed, without passing through the previous necessary steps.

<i>FR-STF-006: Record relevant actions, comments or resolution information.</i>	
<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must require staff to record actions taken, comments, or resolution details when modifying a request or transitioning its status.
<i>Acceptance Criteria</i>	➔ Verify that transitioning a request status prompts the staff member to fill in mandatory comments. ➔ Verify that actions and comments are recorded and added to a read-only log with the user's ID and timestamp.

<i>FR-STF-007: Authorized Request Resolution and Closure</i>	
<i>Stakeholder Source</i>	Authorized Staff
<i>Priority</i>	High
<i>Description</i>	The system must permit only authorized staff members to mark a request as resolved or closed.
<i>Acceptance Criteria</i>	➔ Verify that options to transition a request to resolved or closed are only enabled for users with authorized staff roles.

3. Management and Oversight Capabilities (FR-MNG)

The following requirements govern the capabilities of managers and oversight officers evaluating service performance and operational trends.

<i>FR-MNG-001: Service Activity Dashboard</i>	
<i>Stakeholder Source</i>	Management
<i>Priority</i>	Medium
<i>Description</i>	The system must provide a dashboard containing summarized real-time service activity information, including active requests and resolution rates.
<i>Acceptance Criteria</i>	➔ Verify that the dashboard displays dynamic visual summaries or a tally of service requests. ➔ Verify that dashboard data updates upon page refresh to reflect current up-to-date system records.

FR-MNG-002: Request Tracking

<i>Stakeholder Source</i>	Management
<i>Priority</i>	High
<i>Description</i>	The system must enable managers to immediately identify, filter, and track requests labelled as open, overdue, resolved, and closed.
<i>Acceptance Criteria</i>	➔ Verify that requests that exceed the deadline of resolving them, are flagged as overdue. ➔ Verify that managers can filter request listings specifically for an overdue status across all departments.

FR-MNG-003: View Request Information Through Multi-Dimensional Means

<i>Stakeholder Source</i>	Management
<i>Priority</i>	Medium
<i>Description</i>	The system must allow users to group service requests across multiple dimensions, such as category, status, department, and date ranges.
<i>Acceptance Criteria</i>	➔ Verify that managers can select grouping dimensions from a selection panel and can then see correctly grouped results.

FR-MNG-004: Accountability and Service Performance Capabilities

<i>Stakeholder Source</i>	Management
<i>Priority</i>	High
<i>Description</i>	The system must allow managers to export detailed performance metrics, historical logs, and request information to standard structured formats to support external performance analysis and auditing.
<i>Acceptance Criteria</i>	➔ Verify that managers can export filtered request records. ➔ Verify that the resulting export contains the complete lifecycle of a request with timestamps, staff assignees, comments, and resolution indicators to ensure accountability.

Non-Functional Requirements (NFR)

1. Security Engineering (NFR-SEC)

Security is treated as a lifecycle-wide engineering responsibility rather than a post-development implementation.

NFR-SEC-001: Server-Side Role-Based Access Control

<i>Stakeholder Source</i>	Requester / Staff / Management
<i>Priority</i>	High
<i>Description</i>	The system must enforce server-side access controls to restrict actions and resource access based on authenticated user roles.
<i>Acceptance Criteria</i>	➔ Verify that unauthenticated attempts to access any dashboard or administrative page returns an “Access Denied” error code. ➔ Verify that a user authenticated with the “Requester” role cannot access, retrieve, or update records belonging to other requesters or access any information that only authorized staff and management can view. ➔ Verify that 2 factor Authentication works as intended and provides the necessary security with the use of login keys or security codes.

NFR-SEC-002: Decoupled Secrets and Credential Management

<i>Stakeholder Source</i>	GitHub Governance
<i>Priority</i>	High
<i>Description</i>	Database connection strings, API tokens, passwords, and private keys must never be hardcoded or committed to the repository. They must be loaded dynamically from the runtime environment.
<i>Acceptance Criteria</i>	➔ Verify that the source code uses a structured configuration loader to access connection strings, such as a .env file. ➔ Verify that the project repository includes a .gitignore file that explicitly excludes configuration files or secret files. ➔ Verify that user’s password login credentials are kept safe through password hashing.

NFR-SEC-003: Server-Side Input Validation and Sanitization

<i>Stakeholder Source</i>	Security Standard
<i>Priority</i>	High

<i>Description</i>	All incoming data from a form submitted by a user or API requests must be validated and sanitized on the server side before processing to prevent any unauthorized actions, such as SQL injections and malicious script execution.
<i>Acceptance Criteria</i>	➔ Verify that SQL syntax or execution patterns in fields, such as title, are sanitized which would cause the query to execute purely as literal strings. ➔ Verify that inputs containing HTLP or script tags are stripped before storage. ➔ Verify that any request exceeding predefined size constraints is rejected on the server side.

2. Performance, Reliability, and Scalability (NFR-PRS)

These requirements ensure the system remains responsive, stable, and highly available under typical usage conditions.

NFR-PRS-001: Operational Transaction Response Time

<i>Stakeholder Source</i>	Requester / Staff / Management
<i>Priority</i>	Medium
<i>Description</i>	Standard database retrieval operations and page transitions must load rapidly to prevent users from experience slow system responses and operational delays.
<i>Acceptance Criteria</i>	➔ Verify that 95% of standard read requests complete within 5-10 seconds when tested under a standard baseline load of 10 concurrent active users. ➔ Verify that database write operations complete within 5-10 seconds under the same load conditions.

NFR-PRS-002: High Availability / System Uptime

<i>Stakeholder Source</i>	Requester / Staff / Management
<i>Priority</i>	Medium
<i>Description</i>	The system must remain consistently accessible to stakeholders, minimizing unscheduled maintenance windows.
<i>Acceptance Criteria</i>	➔ Verify that the hosting platform displays zero unscheduled downtime during critical system operations.

3. Usability, Accessibility, and Portability (NFR-UAP)

These guidelines ensure the platform remains inclusive, user-friendly, and compatible with common consumer hardware.

NFR-UAP-001: Cross-Device Mobile Responsiveness

<i>Stakeholder Source</i>	Requester / Staff / Management
<i>Priority</i>	High
<i>Description</i>	The user interface must dynamically scale across standard mobile, tablet, and desktop viewports.
<i>Acceptance Criteria</i>	➔ Verify that all pages render correctly, with zero text overlapping, broken layouts, required horizontal scrolling, by utilizing on screen widths ranging from 320px up to 1920px.

4. Environment, Deployment, and Operations (NFR-EDO)

These requirements dictate the environment boundary controls and ensure a release of engineering protocols necessary to achieve stable, repeatable software delivery.

NFR-EDO-001: Deployment Environment Isolation

<i>Stakeholder Source</i>	Environments and Deployment
<i>Priority</i>	High
<i>Description</i>	The team must maintain separate, fully isolated environments for development, testing, staging, and production.
<i>Acceptance Criteria</i>	➔ Verify that the development database contains only dummy data. ➔ Verify that the staging database is hosted separately from the production database, so that staging tests do not modify production records. ➔ Verify that environment configurations are completely separate and injected through automated environment variables.

NFR-EDO-002: Structured Observability and Event Logging

<i>Stakeholder Source</i>	Operational Readiness
<i>Priority</i>	Medium
<i>Description</i>	The system must record detailed, structured operational logs for critical security events, system exceptions, and request status transitions to facilitate quick troubleshooting and active auditing.
<i>Acceptance Criteria</i>	➔ Verify that all request status transitions, user authentications, and unhandled system

- exceptions generate structured application logs.
- Verify that each log entry includes a clear timestamp, user ID, event type, and relevant exception stack traces in a standard format.

NFR-EDO-003: Rapid Environment Rollback Capability

<i>Stakeholder Source</i>	Deployment and Operations
<i>Priority</i>	High
<i>Description</i>	The hosting platform and deployment pipeline must support rolling back a failed deployment to the previous stable build quickly to ensure the service to continue operating.
<i>Acceptance Criteria</i>	→ Verify that a rolling back command or platform trigger restores the application server and its configuration to the previously deployed release candidate within 5 minutes of a deployment failure.

Constraints

1. The team size is 3 members.
2. The system must be completed by 18 October 2026, with milestones due on 9 September 2026, 30 September 2026 and 14 October 2026
3. The current funding for this project covers only developer salaries however if the project needs funds beyond such it is expected to come out of developer salaries.
4. Role-based access should implement and prevent anyone from accessing stuff they shouldn't.
5. No unwanted WhatsApp accounts that messages should not affected or trigger the platform.
6. The quality should meet the stakeholder requirement and marking rubric.
7. The data they should be stored and used should follow the POPI ACT.

One of the biggest trade-offs is, that with the small funding will lead the team to find cheaper software to use, to prevent from having to use most of the salary. This can lead to weaker and worse software from being used.

Initial RTM

Request Number	Source	Design/Feature	Priority	Status	Verification
FR-REQ-001	Requester	Service Request Submission	High	Not completed	Unit Testing
FR-REQ-002	Requester	Controlled Category Classification	High	Not completed	Unit Testing
FR-REQ-003	Requester	Real-Time Status Visibility	High	Not completed	System Testing
FR-REQ-004	Requester	Request History	Medium	Not completed	System Testing
FR-REQ-005	Requester	Feedback Notifications	Medium	Not completed	Integration Testing
FR-REQ-006	Requester	Service Request Submission through WhatsApp and SMS	High	Not completed	Unit Testing
FR-STF-001	Authorized Staff	Role-Based Service Request Filtering	High	Not completed	Integration Testing
FR-STF-002	Authorized Staff	Advanced Search, Filter, and Sort	High	Not completed	System Testing
FR-STF-003	Authorized Staff	View Request Information	High	Not completed	System Testing
FR-STF-004	Authorized Staff	Responsibility Assignment and Ownership	High	Not completed	System Testing
FR-STF-005	Authorized Staff	Controlled Status Transitions	High	Not completed	Unit Testing
FR-STF-006	Authorized Staff	Record relevant actions, comments or resolution information	High	Not completed	Integration Testing
FR-STF-007	Authorized Staff	Authorized Request Resolution and Closure	High	Not completed	Unit Testing

FR-MNG-001	Management	Service Activity Dashboard	Medium	Not completed	System Testing
FR-MNG-002	Management	Request Tracking	High	Not completed	Unit Testing
FR-MNG-003	Management	View Request Information Through Multi-Dimensional Means	Medium	Not completed	System Testing
FR-MNG-004	Management	Accountability and Service Performance Capabilities	High	Not completed	Unit Testing
NFR-SEC-001	Requester / Staff / Management	Server-Side Role-Based Access Control	High	Not completed	Security Testing
NFR-SEC-002	GitHub Governance	Decoupled Secrets and Credential Management	High	Not completed	Security Testing
NFR-SEC-003	Security Standard	Server-Side Input Validation and Sanitization	High	Not completed	Security Testing
NFR-PRS-001	Requester / Staff / Management	Operational Transaction Response Time	Medium	Not completed	Performance Testing
NFR-PRS-002	Requester / Staff / Management	High Availability / System Uptime	Medium	Not completed	Performance Testing
NFR-UAP-001	Requester / Staff / Management	Cross-Device Mobile Responsiveness	High	Not completed	Compatibility testing
NFR-EDO-001	Environments & Deployment	Deployment Environment Isolation	High	Not completed	Compatibility testing
NFR-EDO-002	Operational Readiness	Structured Observability and Event Logging	Medium	Not completed	Performance testing
NFR-EDO-003	Deployment and Operations	Rapid Environment	High	Not completed	Performance Testing

		Rollback Capability			
--	--	------------------------	--	--	--

Risk Register

Risk Assessment Matrix

Probability	Impact				
	Low (1)	Medium (2)	High (3)	Critical (4)	
	Low (1)	1	2	3	4
	Medium (2)	2	4	6	8
	High (3)	3	6	9	12
	Very High (4)	4	8	12	16

Probability x Impact (RL) determines the priority level:

Priority Level	Risk Level (RL) Range	Description
Critical	RL > 9	Requires immediate, proactive mitigation and daily tracking.
High	4 < RL <= 9	Requires formal mitigation plans and weekly review.
Medium	2 < RL <= 4	Requires documented mitigation and monitored at milestone reviews
Low	RL <= 2	Acceptable residual risk and tracked on the backlog.

Risk ID	Description	Cause	Prob.	Impact	Priority	Mitigation	Contingency	Owner	Status
RSK-001: Technology Stack Learning Curve	The team experiences delays in implementing core business requirements due to a steep learning curve associated with frameworks that may be new to the team.	Selecting modern but unfamiliar technologies without prior engagement with the technologies, underestimating the time required to build proficiency with it.	High	High	High	⇨ Perform time-boxed experimental prototyping to prove basic CRUD compatibility. ⇨ Select technologies with extensive documentation, a strong community support, and sufficient testing libraries.	If the team fails to establish a proficient enough baseline for the technology stack, formally rollback the technology choice to a simpler, familiar stack through an Architecture Decision Record (ADR).	Stephanus Cornelius Zondagh	Open
RSK-002: GitHub Pull Request Bottlenecks	Merging important functional code to the main branch is delayed, leading to potentially missing deadlines.	Enforcement of mandatory GitHub governance policies that requires two independent peer approvals before any merge.	High	High	High	⇨ Enforce an agreement mandating a 48-hour time limit in reviewing assigned peer reviews (PR). ⇨ Decompose features into small, incremental additions to make reviews highly focused and of lower effort. ⇨ Link every PR directly to an active task or issue on the GitHub project board.	Organize an immediate team peer review session if more than three PRs remain unreviewed for over 3 days, performing joint walkthroughs and resolving conflicts live.	Liam Dickson	Open
RSK-003: Public Exposure of Secrets and Credentials	Unauthorized users gain access to private keys, database passwords, or API tokens, compromising platform security.	Committing configuration files containing hardcoded credentials directly into the public GitHub repository during rapid local development.	Medium	Critical	High	⇨ Implement and test a .gitignore configuration before committing any project directories. ⇨ Store all environment-specific secrets locally and manage them on hosting platforms using environment variables.	If a leak occurs, immediately revoke and cycle the leaked credentials on all hosting environments, configure the GitHub repository history to remove the secret, and document the security risk in the PED.	Liam Dickson	Open

RSK-004: Environment Drift Between Staging and Local Setups	Code that functions correctly during local developer testing fails to build, integrate, or run once deployed to staging or production.	Significant differences among local developer operating systems, database engines, or runtime environment versions.	Medium	High	High	⇒ Deploy a baseline application to the staging environment to verify the CD pipeline. ⇒ Enforce identical runtime environments across all team local machines using system version-locking files, such as a package-lock.json	Execute a rollback procedure to the last verified stable build on staging, analyze the failing build's error logs, and recreate the staging environment configuration locally for targeted debugging.	Stephanus Cornelius Zondagh	Open
RSK-005: Uncontrolled Scope Creep	The project team runs out of time for development and testing due to adding out-of-scope features, risking a failure to deliver core functionality.	Designing and implementing additional features that are out-of-scope to boost project appeal without performing a formal impact analysis or mapping the feature to the original constraints.	Medium	Medium	Medium	⇒ Map every active task to a baselined functional requirement in the RTM. ⇒ Decline or defer any feature request that is not specified by the core business capabilities.	Perform a formal change request and impact analysis to separate the unapproved feature and move it to future scope in the PED, focusing on restoring and updating the core functional baseline.	Stephanus Cornelius Zondagh	Open
RSK-006: AI-Generated Code Verification Failure	The system experiences runtime errors that are hard to debug, logic flaws, or security vulnerabilities that the team is unable to diagnose.	Relying on AI tools to write complex application logic, algorithms, or unit tests, and merging the output without performing code walkthroughs or regression tests.	Medium	Medium	Medium	⇒ Enforce the practice of logging every AI contribution in the AI Usage Register with documentation of how the team verified the output. ⇒ Write unit and integration tests before using AI to write code, forcing the AI output to pass re-established quality requirements. ⇒ Treat all AI generated code as unverified and perform code reviews.	If an AI-assisted component fails testing and cannot be explained during a walkthrough, isolate the component and reengineer the feature manually.	Ruan Wolmarans	Open
RSK-007: Free-Tier Hosted	The software's deployment platform halts	Frequent CI/CD build pipelines resulting in rigid	Low	High	Medium	⇒ Perform a platform resource review	If a cloud service is suspended	Liam Dickson	Open

Platform Limit	the staging phase mid-lifecycle, causing a loss to continuous testing and impacting schedule constraints.	resource thresholds of free-tier cloud providers being exceeded.				to document free-tier limits, storage capacities, and baseline runtimes. ⇒ Configure the CI test pipeline to run on a local database rather than querying the cloud deployment platform each time.	due to exceeding limits, temporarily halt non-essential monitoring services, perform local integration testing, and utilize alternative backup hosting services while documenting the operational constraint in the PED.		
RSK-008: Notification Loss	Pending SMS/Email notifications are lost before dispatch	Notifications are dispatched asynchronously inside the API process within a queue that is not persistent, and so when Render restarts or spins down the service, the notification gets lost.	Medium	Medium	Medium	Log every dispatch outcome so missing notifications are detectable	Staff resend from request history.	-	Open
RSK-009: Render Spin-Down	Requests, notifications and WhatsApp webhooks are delayed or fail while the service starts up again.	Render free web services spin down after 15 idle minutes and take about one minute to start up again.	High	High	High	Start the production up again before demonstrations. Monitor webhook failures in logs.	Move production to a paid Render instance	-	Open
RSK-010: Storage Cap and Backups	Database writes fail or data cannot be recovered	Atlas Free Cluster is capped at 512 MB and has no automated backups.	Medium	High	High	⇒ Monitor database size. Schedule exports of production.	Restore from the latest export. Archive dummy data. Upgrade the cluster through change control.	-	Open
RSK-011: Trial Expiry	SMS or Email notification stop before the final demo.	Twilio trial expires after 30 days or when credit runs out, the SendGrid trial stops	Medium	High	High	⇒ Record account creation dates so expiry falls after the project demo, monitor credit and no	Demonstrate using a mocked provider and dispatch logs.	-	Open

		sending after 60 days and is available only once.				load testing through providers.			
RSK-012: Restricted Recipients	Real Requesters cannot receive notifications or use WhatsApp intake.	Trial and test accounts deliver only to verified or pre-approved recipients. (Twilio has a maximum of 5)	Very High	Medium	High	⇒ Team members registered as demo requesters. Documented as a demonstration constraint.	Demonstrate using dispatch logs. Upgrade accounts if funded.	-	Open
RSK-013: WhatsApp Reply Window	Status updates cannot be sent by WhatsApp after the conversation window closes.	Free-form replies are only allowed within 24 hours of the requester's last message.	Medium	Medium	Medium	⇒ Acknowledge each request immediately on intake. Send status updates later through SMS/Email	Revisit message templates once a registered business number exists.	-	Open

Architecturally Significant Requirements and Quality Drivers

A requirement was determined as an Architecturally Significant Requirement (ASR) when it had a high architectural impact coinciding with either high stakeholder value or business value or a direct link to a project constraint or open risk. These candidates were chosen after reviewing the PED v1.0 requirements, constraints, risk register and engineering consideration decisions.

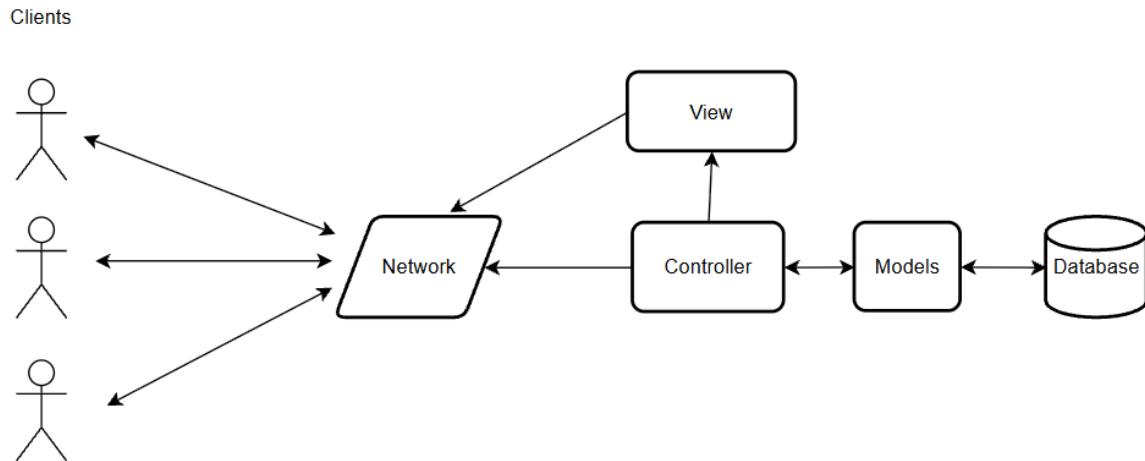
ASR (Rank – Value/Impact)	Scenario	Response Measure	Traces To	Architectural Influence
ASR-001: Authorization and Data Ownership (1 – High/High)	A user bypasses authentication requirements and makes a request that they are not authorized for. A required response is that the request is rejected before any data is changed or	All protected endpoints are enforced with server-side verification along with tests that return valid response codes, such as 401 when the user is	Community Members, NFR-SEC-001, FR-STF-001, FR-REQ-004, FR-STF-007, Constraints 4 & 7, ECD_0001, RSK-003	Authentication and authorization must be served through the backend before core business logic. Ownership should be assigned on every request. Express

	read, based on the user's role, department or ownership.	unauthenticated. Logins are rejected without a valid second factor authentication (2FA).		middleware used for authorization.
ASR-002: Lifecycle Integrity and Auditability (2 – High/High)	Staff change the status of a request, assign a new request, or record any information on a request. The required response to this is that only valid downstream transitions are accepted, comments become mandatory, and the status is saved automatically. The history of requests can be obtained.	No invalid transitions in tests with a history entry per transition. FR-MNG-004 outputs an export that contains the complete ordered lifecycle.	Management and Community Engagement Teams, FR-STF-004 to FR-STF-007, FR-MNG-004, NFR-EDO-002	Transition rules isolated in the domain layer. MongoDB document atomicity.
ASR-005: Deployability within cost (3 – High/High)	Team deploys a release or a deployment fails. The required response to this is that the system runs on a free hosting platform with configuration and secret credentials being handled via environment variables, with rollback being	Cost within Constraint 3, No secrets being stored in public repository, rollback takes less than 5 minutes (NFR-EDO-003)	Project team, constraint 3, ECD_0002, NFR-EDO-001/003, NFR-SEC-002, NFR-PRS-002, RSK-003/004/007	Few deployable units with a stateless backend, where each request is treated independently with no user session data being stored in between requests. Node/Express and Angular build deployable on free-tier hosting platform with a managed MongoDB.

	available for deployment failures.			
ASR-003: Modifiability (4 - Medium/High)	A new notification system or workflow change is requested after baseline decisions and so the required response is to limit the change to the affected module.	New notification system needs only a new Observer/Adapter and its registration to it. Peer-Reviews are also undertaken to ensure that there is no unnecessary additional changes.	Community Engagement Teams, ECD_0003, RSK-005	Defined module boundaries such as requests, notifications, authentication, and reporting.
ASR-004: Notification Timeliness and Fault Isolation (5 - Medium/Medium)	Staff perform a status change, and the notification provider is slow or experiencing downtime. The required response is that the notification is dispatched, any provider failures are logged and never blocks or rolls back the change.	When a request change is committed, the provider handles the notification and dispatches it within 10 seconds. A simulated failure leaves the change committed with a structured log entry.	Requesters, FR-REQ-005, NFR-EDO-002, Constraint 3, external provider dependency	Asynchronous post-commit dispatching. The use of Node for non-blocking I/O for notification dispatching.

Architecture decision and diagrams

For the CivicConnect platform, we will be using a Client-Server architecture and Model-View-Controller (MVC) architecture. We are choosing these architectures, because it allows us to enforced server-side verification and it also allows us to keep the interfaces, and back-end split to help with modularity and scalability of the platform for if the stakeholder wants more features.



Architecture is different than technologies and logical layers, because architecture would give the framework of the platform while the technologies would help give functions and features to the platform. The physical deployment tiers is the platform working using multiple technologies to function.

Technology-stack decision

For the CivicConnect platform, we are going to use Angular (v22) for the frontend, with Express (v5.2.1) and Node.js (v26.10.0) for the middleware, with JavaScript (IntelliJ IDEA 2026.1.4) as the backend and MongoDB as our persistence. We are using this stack because the whole team is well versed with most of the stack, except for Angular. However Angular was still chosen, because how it integrates well with the rest of the stack, as well its own testing to help with verifications. It was always chosen for it is being a free open-source stack that help limit development costs and helps with dependency and integration as there will be a lot of free APIs that can help with the platform. This stack also modular which will help with maintaining the platform and making it easier to make it compatible with any kind of deployment we want to do with it. Security is our only risk that we must put in more effort in the coding phase to help prevent any security risks.

There will be future dependency:

- Official WhatsApp Business Platform API, which has compatibility with Node.js.
- Twilio which has compatibility with Node.js.

Data and persistence baseline

Mongo DB Schemas

Users:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["username", "password_hash", "first_name", "last_name", "clearance"],
      properties: {
        username: { bsonType: "string" },
        password_hash: { bsonType: "string" },
        first_name: { bsonType: "string" },
        last_name: { bsonType: "string" },
        clearance: { bsonType: "int", minimum: 0, maximum: 5 },
        work_categories: { bsonType: "array", items: { bsonType: "string" } },
        email: { bsonType: ["string", "null"], pattern: "^.+@.+\\.\\..+$" },
        phone: { bsonType: ["string", "null"], pattern: "^[0-9]{10}$" },
        whatsapp: { bsonType: ["string", "null"], pattern: "^[0-9]{10}$" }
      }
    }
  }
});
```

// Indexes

```
db.users.createIndex({ username: 1 }, { unique: true });
db.users.createIndex({ email: 1 }, { unique: true, sparse: true });
```

Requests:

```
db.createCollection("requests", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["details", "requesters", "security_level", "request_category", "address", "active"],
```

```
properties: {
  group_id: { bsonType: ["objectId", "null"] },
  details: { bsonType: "string" },
  requesters: {
    bsonType: "array",
    items: { bsonType: "objectId" },
    description: "Array of ObjectIds referencing the users collection"
  },
  security_level: { bsonType: "int", minimum: 0, maximum: 5 },
  request_category: { bsonType: "array", items: { bsonType: "string" } },
  address: { bsonType: "string" },
  active: { bsonType: "bool" },
  notes: {
    bsonType: "array",
    items: {
      bsonType: "object",
      required: ["note_id", "details", "category", "user_id", "date"],
      properties: {
        note_id: { bsonType: "objectId" },
        details: { bsonType: "string" },
        category: { bsonType: "string" },
        user_id: { bsonType: "objectId" },
        date: { bsonType: "date" }
      }
    }
  }
}
});
```

```
// Indexes for common access patterns

db.requests.createIndex({ active: 1, security_level: 1 });

db.requests.createIndex({ group_id: 1 }, { sparse: true });

db.requests.createIndex({ requesters: 1 });
```

Request_Groups:

```
db.createCollection("request_groups", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["created_at", "created_by", "status"],
      properties: {
        created_at: { bsonType: "date" },
        created_by: { bsonType: "objectId" },
        status: { enum: ["OPEN", "IN_PROGRESS", "RESOLVED"] },
        title: { bsonType: "string" },
        primary_request_id: { bsonType: "objectId" } // The anchor request
      }
    }
  }
});
```

LOGS

```
db.createCollection("system_logs", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["log_id", "timestamp", "event_type", "severity", "actor", "action"],
      properties: {
        log_id: { bsonType: "objectId" },
        timestamp: { bsonType: "date" },
```

```
event_type: {
  enum: ["USER_AUTH", "REQUEST_LIFECYCLE", "NOTE_CREATED", "GROUP_ASSIGNED",
"SECURITY_CHANGE", "SYSTEM_ERROR"],
  description: "Must be a recognized domain event category"
},
severity: {
  enum: ["INFO", "WARNING", "ERROR", "CRITICAL"],
  description: "Log urgency level"
},
actor: {
  bsonType: "object",
  required: ["user_id", "username", "clearance_at_time"],
  properties: {
    user_id: { bsonType: "objectId" },
    username: { bsonType: "string" },
    clearance_at_time: { bsonType: "int", minimum: 0, maximum: 5 },
    ip_address: { bsonType: ["string", "null"] }
  }
},
target_entity: {
  bsonType: "object",
  required: ["entity_type", "entity_id"],
  properties: {
    entity_type: { enum: ["REQUEST", "USER", "GROUP", "NOTE"] },
    entity_id: { bsonType: "objectId" }
  }
},
action: { bsonType: "string" },
changes: {
  bsonType: ["object", "null"],
  description: "Captures before and after states for security or status audits",
```

```

    properties: {
      field: { bsonType: "string" },
      old_value: { bsonType: ["string", "int", "bool", "null"] },
      new_value: { bsonType: ["string", "int", "bool", "null"] }
    },
    metadata: {
      bsonType: ["object", "null"],
      description: "Flexible key-value payload for contextual details"
    }
  }
}
});

// INDEXES FOR AUDIT AND TIME-SERIES PERFORMANCE

db.system_logs.createIndex({ timestamp: -1 });

db.system_logs.createIndex({ "target_entity.entity_id": 1, timestamp: -1 });

db.system_logs.createIndex({ "actor.user_id": 1, timestamp: -1 });

db.system_logs.createIndex({ timestamp: 1 }, { expireAfterSeconds: 31536000 });

```

Justification and Persistence model:

Category	Assessment & Recommendations
Structure & Embedded Notes	Embedding Notes inside the Request document follows MongoDB's design principles. Through this it is found that when fetching a request all related notes automatically get returned in a single read operation without requiring \$lookup joins.

Category	Assessment & Recommendations
BSON 16MB Limit & Document Growth	An accepted risk of MongoDB is unbounded document growth. However, because notes are short maintenance records, its found that a single request would require more notes than it is foreseeably possible to aquire to approach the 16MB BSON limit. This risk is therefore very low for standard usage.
Access Patterns	Read-heavy dashboard views filtered using the active property would benefit heavily from compound indexes ({ active: 1, security_level: 1 }).
Referential Integrity	MongoDB does not enforce foreign key constraints natively. If a user ID is deleted from the users collection, application code or middleware must ensure orphan references aren't left in requesters or notes.user_id.
Data Consistency & Duplicate Notes	Writing duplicate notes across multiple grouped requests requires multi-document transactions or MongoDB \$position / \$push bulk updates inside a single session.

Scalability Strategy (Sharding)

- **Shard Key Selection:** If the database grows beyond a single replica set, shard the requests collection using a compound shard key:

Shard Key = {active: 1, _id: 1}

This ensures write operations are distributed while keeping queries for active maintenance requests targetable to specific shards.

Concurrency, Validation, and Transaction Integrity

Multi-Document ACID Transactions:

Integration of MongoDB session transactions in order to ensure all transactions meet ACID requirements.

Example for a Note that needs to be saved to multiple transactions.

```
const session = db.getMongo().startSession();
```

```
session.startTransaction();
```

```
try {  
  const notePayload = {  
    note_id: new ObjectId(),  
    details: "Investigated water shutoff valve. Main breaker reset.",  
    category: "Engineering",  
    user_id: ObjectId("65f1a2b3c4d5e6f7a8b9c0d1"),  
    date: new Date()  
  };  
  
  // Push note to all active requests belonging to the target group  
  db.requests.updateMany(  
    { group_id: ObjectId("65f99999999999999999999999999999"), active: true },  
    { $push: { notes: notePayload } },  
    { session }  
  );  
  
  session.commitTransaction();  
} catch (error) {  
  session.abortTransaction();  
  throw error;  
} finally {  
  session.endSession();  
}
```

Security and Clearance Enforcement:

Authorisation Checks

Preform authorisation checks at the application level before running transactions.

e.g:

```
User.clearance >= Request.security_level
```

User Roles:

User roles beyond clearance are stored in the work categories field and additionally dictate what actions can be taken and what objects a user can act upon

Field-Level Encryption

Encryption of key fields like User.phone or User.Email deferred till implementation viability has been established.

Password Hashing:

All stored passwords will be hashed to protect against leaking in the case of a database security breach.

Traceability artifacts.

Req ID	Description	Component / Schema	Validation / Test Strategy
REQ-DAT-001	Categorize requests	requests.request_category	JSON Schema validation
REQ-DAT-002	Grouped note replication	db.requests.updateMany + Transactions	Integration test verifying atomic rollback
REQ-SEC-001	Clearance enforcement	Application middleware + system_logs	Audit log event generation on clearance access failure

Req ID	Description	Component / Schema	Validation / Test Strategy
REQ-SEC-002	Password protection	users.password_hash	Unit test enforcing Argon2id hashing prior to insert

Initial Design Decisions — Informed by A2 Research

Design Decision 1:

Category	Design Decision 1: Notification Feedback Logic
Problem Context & Design Problem	Tight Coupling in Status Feedback Logic: CivicConnect is expected to send notifications (email/SMS/ Whatsapp messages) to requesters within 10 seconds of a request update. However, it would violate SRP and create Tight coupling with third part API's if notification logic were integrated directly inside the Request-Update component.
A2 Research Evidence & Alternatives Considered	• Candidate A (Observer Pattern): Define a one-to-many dependency where status changes emit events to notify dependent observers asynchronously. • Candidate B (Strategy Pattern): Encapsulate notification algorithms behind a common interface and execute dynamically. • A2 Recommendation: Observer Pattern (Khanaferov, 2013) to decouple request domain logic from external messaging platforms.

Category	Design Decision 1: Notification Feedback Logic
Final Pattern / Approach Selection & Justification	Observer Pattern (Event-Driven Observer) Upon the updating of a request, the RequestSubject emits domain events (RequestStatusChanged) without knowing the specific delivery mechanism. Dependent observers like EmailNotifier or SMSNotifier would react asynchronously to fulfill the time requirement. This method will completely decouple request updates from third-party notification APIs (e.g., WhatsApp/Twilio), satisfying SRP and OCP.
Expected Benefits & Introduced Trade-offs	• Benefits: Lowers structural coupling, maximizes cohesion, allows independent testing of status logic without invoking external APIs, and simplifies adding new channel observers. • Trade-offs: Observer registration and event dispatching introduce minor event plumbing overhead; state synchronization relies on asynchronous event handling.
Affected Modules / Components / Interfaces	• Request (Subject / Event Issuer) • IRequestUpdateObserver (Interface) • EmailNotificationObserver & SMSNotificationObserver (Concrete Observers) • NotificationService

Category	Design Decision 1: Notification Feedback Logic
ADR & Traceability Artifact Links	ADR-003: Asynchronous Notification Feedback Architecture • <i>RTM / FR Links:</i> FR-REQ-005 (Automated Requester Feedback within 10s)
Initial Implementation Evidence	<pre> javascript // Observer Interface & Subject class RequestSubject { constructor() { this.observers = []; } attach(observer) { this.observers.push(observer); } notify(event) { this.observers.forEach(obs => obs.update(event)); } async updateStatus(newStatus) {this.status = newStatus; this.notify({ requestId: this.id, status: newStatus }); }} </pre>

Design Decision 2:

Category	Design Decision 2: Audit Logging & Cross-Cutting Event Pipeline
Problem Context & Design Problem	Coupled System Auditing & High-Cohesion Requirements: To meet the security expectations of the CivicConnect platform it requires continuous recording of system operations (security clearance adjustments, request lifecycle changes, user authentication events, and note

Category	Design Decision 2: Audit Logging & Cross-Cutting Event Pipeline
	attachments) into a log auditing. However, directly embedding log-dispatching logic inside domain status handlers and authentication controllers would introduce tight coupling, reduce cohesion, and violate the SRP. As outlined in the A2 research, attempting to manage status transitions and information recording within a single component creates an increased risk of unpermitted actions and difficult to maintain code.
A2 Research Evidence & Alternatives Considered	• Candidate A (Inline Synchronous / Command Pattern Logging): A2 evaluated the encapsulation of primitive actions or directly invoking database log insertion within business handlers (Zarras, 2020). However, while Command objects decouple invokers from execution, they do not inherently enforce state transition boundaries and additionally require explicit client-side configuration, creating tight structural coupling between domain handlers and logging targets. • Candidate B (Observer Pattern / Event-Driven Pipeline): A2 evaluated the Observer pattern (Khanaferov, 2013) as a method to decouple event producers from observers. The A2 showed that emitting asynchronous events when state changes occur would maximize cohesion, and decouple core logic from secondary integrations. • A2 Recommendation Alignment: The team selected an event-driven architecture to decouple domain actions from downstream side-effects (notifications and audit logging).
Final Pattern / Approach	Observer Pattern (Domain Event Bus Architecture) The domain engine publishes immutable domain events (e.g., RequestLifecycleEvent, SecurityLevelUpdated,

Category	Design Decision 2: Audit Logging & Cross-Cutting Event Pipeline
Selection & Justification	UserAuthenticated) to a central DomainEventBus. Our implementation would leverage independent listeners, such as the AuditLogSubscriber, that subscribe to these events asynchronously to format and commit log entries directly to the system_logs MongoDB collection. This will completely isolate all audit logging mechanics from the core business transactions, thus ensuring domain handlers remain focused purely on state rules.
Expected Benefits & Introduced Trade-offs	• Benefits: Strong decoupling between domain logic and audit infrastructure; high cohesion; allows addition of new audit subscribers (e.g., analytics, real-time security alerts) without modifying existing domain code; simplifies unit testing of business logic. • Trade-offs: Introduces slight event-dispatching latency and will require handling of event delivery ordering and retry logic during database writes.
Affected Modules / Components / Interfaces	• IDomainEventPublisher / DomainEventBus • AuditLogSubscriber • SystemLogRepository • system_logs Collection (actor, target_entity, changes schemas)
ADR & Traceability Artifact Links	ADR-002: Decoupled Event-Driven Audit Logging Engine

Category	Design Decision 2: Audit Logging & Cross-Cutting Event Pipeline
	• <i>RTM / Traceability Links</i>: REQ-SEC-001 (Security Clearance & Audit Enforcement)
Initial Implementation Evidence	<pre>` `` `javascript class DomainEventBus { constructor() { this.subscribers = []; } subscribe(subscriber) { this.subscribers.push(subscriber); } publish(event) { this.subscribers .filter(s => s.supports(event)) .forEach(s => s.handle(event)); } } // Isolated audit subscriber handling system_logs insertions class AuditLogSubscriber { supports(event) { return event.isAuditable; } async handle(event) { await db.collection("system_logs").insertOne({ log_id: new ObjectId(), timestamp: new Date(), event_type: event.type, severity: event.severity, actor: event.actor,</pre>

Category	Design Decision 2: Audit Logging & Cross-Cutting Event Pipeline
	target_entity: event.targetEntity, action: event.action, changes: event.changes

Deployment Compatibility

Hosting must cost R0, conforming to Constraint 3 and ASR-005, so the system is deployed on free tiers.

Physical Tier	Logical Responsibility	Deployment Target
Client	Angular Single Page Application (SPA) running in the browser	Render static site
Application Server	Express REST API: MVC controllers and models	Render free web service
Data	MongoDB	MongoDB Atlas Free Cluster
External Services	WhatsApp intake, SMS and Email notifications	Meta WhatsApp Cloud API, Twilio SMS, Twilio SendGrid email

As for the Environments, development will run locally with dummy data with Staging and Production each having their own Render services and their own Atlas project and their own secrets. Tests run locally or in CI rather than in a separate hosted test environment.

Plausibility of Decisions

Decision	Deployment Implication	Verdict
----------	------------------------	---------

Client-server/MVC with a stateless API	Render may restart or spin down the service at any time	Plausible. No session or business state held only in memory.
MongoDB transactions	Transactions require a replica set and Atlas provides one, a standalone local instance does not.	Plausible. Use a single-node replica set locally.
In-process Observer notifications	Pending SMS/Email are lost if the service restarts or spins down	Plausible but for demonstration and testing purposes.
Render free web service	Wakes in about one minute after being idle for 15 minutes	Plausible. Wake the hosted production before demonstration.
Rollback within 5 minutes	Render free instances support rollback to a previous deploy	Plausible. Test in Staging.
Twilio and SendGrid trials	Verified recipients only, 30-day expiry. SendGrid allows 100 emails/day, stops after 60 days.	Plausible but for demonstration only as per the free/trial availability options.
WhatsApp intake	Needs a public HTTPS webhook, free-form replies only within 24 hours of the requesters last message	Plausible. Acknowledge on intake and later update by SMS/Email.

Configuration, State & Networking

Configurations and secrets are loaded from a .gitignore .env file locally. Secrets would involve the Atlas Connection String, authentication secrets, Twilio Auth Token, SendGrid API key, app secret, and Meta access token.

Furthermore, persistence and state of data is handled via Atlas as Atlas is the only durable store and the system's single point of failure which is accepted at this load. Scheduled exports of production will be needed as backup control as Atlas Free clusters offer no automated backups. Requester phone numbers and emails are processed by Meta and Twilio outside of South Africa, so POPIA cross-border conditions apply as per Constraint 7.

Lastly, networking will utilize HTTPS throughout. Meta must reach the API over public HTTPS with each webhook's signature being validated with the app secret before processing as per Constraint 5.

Deferred Deployment Decisions

Deferred Decision	Evidence Required to Decide
Paid hosting (Render paid instance, larger Atlas Cluster)	Measured spin down and webhook failures, storage limitations with growth, stakeholder availability needs, and funding.
Paid Twilio and SendGrid plans. Registered WhatsApp business number	Expected message volumes, cost per message, and Meta business verification details.

Realized Issues Log

This tracks when a documented risk has materialized and serves as an active link showing how the team responded to it.

Issue ID	Date Materialized	Associated Risk	Impact / Consequence	Resolving Decision	Status
ISS-001	N/A	N/A	N/A	N/A	N/A

Engineering consideration documentation

Engineering decisions must coincide with a decision ID and the completion of one of the tables below:

Decision ID	Category	Decision
	Status	
	Summary of issue	
	Summary of decision	
	Trade off made	
	Impact and context	

Future engineering considerations

<i>Decision ID</i>	<i>Category</i>	<i>Decision</i>
<i>ECD_0001</i>	Status	Approved
	Summary of issue	Security is a primary concern for users.
	Summary of decision	Early implementation of password and user data hashing. Aswell as ensuring roll based access is implemented from the beginning to prevent users from accessing data they should not have access to
	Trade off made	Slows initial development of program in favour of ensuring a shorter security validation stage and better user confidence
	Impact and context	Supports better data control practices and ensures regulatory compliance with legal constraints.

<i>Decision ID</i>	<i>Category</i>	<i>Decision</i>
<i>ECD_0002</i>	Status	Differed
	Summary of issue	System uptime requirements may be infeasible given project budget.
	Summary of decision	Initial deployment set to leverage free hosting tools with uptime limited to work hours of the institution. This programme availability will be revisited when financial viability for hosting is established.
	Trade off made	Uptime sacrificed to ensure project suits running cost constraints
	Impact and context	Minimum stakeholder needs met while budget constraints are maintained.
<i>ECD_0003</i>	Status	Approved
	Summary of issue	Project requirements evolving as a byproduct of stakeholder wants changing throughout the development of the program
	Summary of decision	Official form created for requesting of changes. Protocols set in place for how to direct stakeholders towards this page when making requests.
	Trade off made	Stakeholders may feel frustrated with the added layer between them and the project management team however this frustration does not overshadow the value provided by the traceability for changes requested to the program.
	Impact and context	This extra layer provides a main source for all changes made to the scope of the project and allows for traceability of changes made to the specific stakeholder who requested it. This ensures the chain of accountability for change is unbroken and that the final product is more defensible against scrutiny.

<i>Decision ID</i>	<i>Category</i>	<i>Decision</i>
<i>ECD_0004</i>	Status	Approved
	Summary of issue	Considerations over the programming languages to be used in the development of the program need to be made so that future decisions over tech stack and data handling can be made.
	Summary of decision	The team decided to try stick to known languages in order to best adhere to the time constraints related to the project. This limits the languages we can choose from however enables the team to make decisions backed by better judgement throughout the rest of the project.
	Trade off made	Known languages chosen to ensure time to market is minimized, however program efficiency sacrificed as other languages that may be better for this specific project may exist outside of the teams known languages.
	Impact and context	Limitations to the tech stacks that can be used accepted as decision made to not force the team to learn new languages given the work load the team finds itself under.
<i>ECD_0005</i>	Status	Approved
	Summary of issue	Concerns over the technology stack being chosen late causing issues for identifying risks and key consideration in future.
	Summary of decision	The biggest consideration was to meet team capabilities and speed to market.
	Trade off made	The team's capabilities with the technology stack are well known, however the stack is widely implemented thus security vulnerabilities of the stack are more well known and groups have spent more time targeting this architecture seeking vulnerabilities.
	Impact and context	The team is capable of facilitating the needs of the project using the proposed tech stack. This familiarity will also enable the team to judge future concerns with more in depth knowledge of the limitations of the tech stack chosen.

<i>Decision ID</i>	<i>Category</i>	<i>Decision</i>
<i>ECD_0006</i>	Status	Deferred
	Summary of issue	Data retention and traceability
	Summary of decision	Data retention will be based off many factors, including the type of request, last access time, flagging for importance, and legal needs. These factors will all contribute to how long request data will be held onto however the selection of exact durations has been deferred till the stakeholders have given a better understanding of the legal requirements on their end
	Trade off made	Reduction in costs of data storage at the cost of deleting old data. This reduces the historic completeness of the data.
	Impact and context	Costs are expected to be reduced, but harms may be felt in areas of traceability and analytics. These harms are mitigated by the plethora of criteria that are expected to be implemented to specify the value of the data.

<i>Decision ID</i>	<i>Category</i>	<i>Decision</i>
<i>ECD_0007</i>	Status	Approved
	Summary of issue	Project requirements added for integration of WhatsApp and email request submissions
	Summary of decision	The team has decided on using the official WhatsApp Business Platform's API to help get request submissions from WhatsApp and Twilio for email request submissions. It was also decided that Ruan Wolmaranse will take responsibility over implementing Twilio and Stef Zondagh will take responsibility over implementing the WhatsApp API.
	Trade off made	Platform will increase in complexity and will have dependency on the new APIs
	Impact and context	Cost will increase in the post development phase to handle the subscriptions needed to handle more message requests from both APIs.

Github & Team Governance

Accountability Architecture

All members of the team are responsible for ensuring all merges to the main branch are validated. In order to promote this a simple hierarchy has been set up.

Member type	Role	Explanation
Team lead	Verify all code being merged into main has been thoroughly reviewed and enforce review processes	Must ensure all development branches remain functional regardless of project timelines. It is also this member's role to ensure branch names, push titles, and push descriptions are up to the programming standards as set out by the coding standards document.
Feature Lead	Develop a feature or section or code that meets quality standards.	The feature lead is given a section of code to develop (a Feature) and are

		expected to implement that feature while adhering to best coding practices as well as the standards set out by the coding standards document.
Feature Reviewer	Ensure code that has been developed is tested before it is submitted to be added to the main branch	Required to sign for and review all features before they are merged into development branches. These reviews must ensure full functionality of the feature with the version of the program it was developed for.

Github security policy

Branch rules

- Main branches:
 - Sign off by 1 fellow team member
 - Approval by team lead
- Development branches:
 - Sign off by 1 fellow team member
 - All comments/ concerns from all reviews are resolved
 - Further updates to code reset the review process
- Implementation branches:
 - No sign off needed
 - All pushes expected to be well titled with an adequate description of changes made
- Revision branches:
 - No sign off needed
 - All pushes expected to be well titled with an adequate description of changes made

AI usage and accountability

AI in the modern age is a tool that has fundamentally changed the way software developers work on projects. However, it is vital that we do not accept these tools outputs without properly validating their work. What follows is an outline for how we plan to keep the users of AI tools accountable and how we expect these processes to be used.

AI usage document

<i>Criteria</i>	<i>Description</i>
<i>AI Tool Used</i>	The name of the AI tool used (e.g. Grok, ChatGPT, ect.)
<i>Where was output used</i>	Where in the program was the output used. Be specific. (e.g. Code used in GetName function in class Pets.cs on branch Development/Pets/Security)
<i>Reason for usage of tool</i>	Give reason tool use is justified for this area of work and factors that made this necessary. (Time constraints for completion of this feature combined with simplicity of task justifies the implementation of AI tools to speed up completion time)
<i>How was output verified</i>	Give methodology for the verification of the AI output to ensure its output was valid and adequately fulfilled the needs of the request. Additionally include issues detected. (e.g. Thoroughly read through the code to ensure implementation matched the needs of the feature, concerns over coding practices raised based off tools excessive usage of for loops. Testing of feature returns no errors of output issues.)
<i>Output used</i>	Yes/ No
<i>Signed off by</i>	Name of team member who signed off on the push to main development branch that contained this output. "N/A" to be used if output was not used or document is for a minor push to a feature development branch that is not signed off on.

Usage methodology

Complete one table per class where AI outputs were used. These Tables need to be completed before the code is pushed to its branch. One Document per push where AI was used and merge all documents from a branch into a master document that contains all usage tables when that branch is terminated or pulled into another branch. E.g. when a development branch is pulled into main all AI usage documents from that development branch must be combined into one document and saved to the main branch to ensure AI implementation is traceable to its origine and as a pre-emptive paperwork limitation protocol.

Finalized By:

Name: Stef Zondagh

Date: 30-09-2026

Approved By:

Name: Liam Dickson

Date: 30-09-2026